

蓝桥杯30天算法冲刺集训



Day-19 树状数组与ST表

① Note

本讲义内容基本都可以用线段树去实现，但是线段树写起来有点麻烦常数还大，所以还是需要学一学别的方法的。

树状数组

基本思想

二进制唯一分解 + 前缀和

根据任意正整数关于 2 的不重复次幂的唯一分解性质，若一个正整数 x 的二进制表示为 10101，其中等于 1 的位是 0, 2, 4，则正整数 x 可以被“二进制分解”成 $2^4 + 2^2 + 2^0$ 。进一步地，区间 $[1, x]$ 可以分成 $O(\log x)$ 个小区间：

- 长度为 2^4 的小区间 $[1, 2^4]$ 。
- 长度为 2^2 的小区间 $[2^4 + 1, 2^4 + 2^2]$ 。
- 长度为 2^0 的小区间 $[2^4 + 2^2 + 1, 2^4 + 2^2 + 2^2 + 2^0]$ 。

树状数组就是一种基于上述思想的数据结构，其基本用途是 **维护序列的前缀和**。对于区间 $[1, x]$ ，树状数组将其分为 $\log x$ 个子区间，从而满足快速询问区间和。

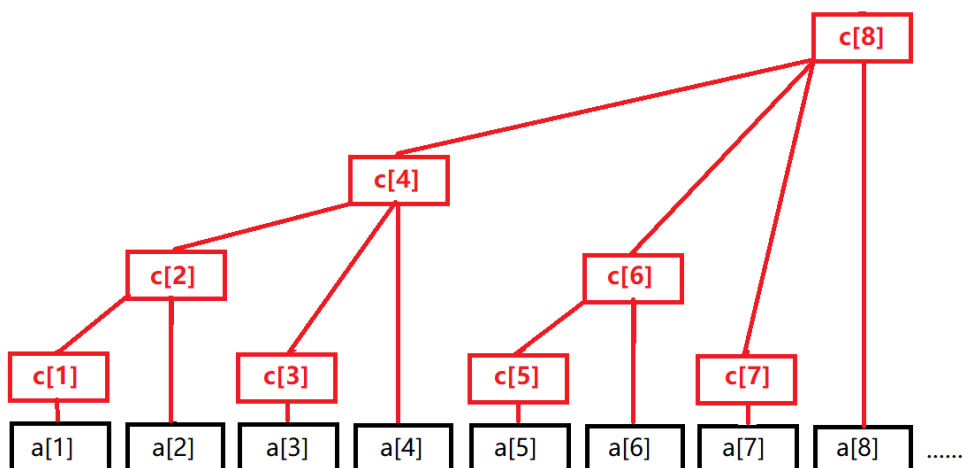
基本操作

由上文可知，这些子区间的共同特点是：若区间结尾为 R ，则区间长度就等于 R 的“二进制分解”下最小的 2 的次幂，我们设为 $\text{lowbit}(R)$ 。

对于给定的序列 $a[]$ ，我们建立一个数组 c ，其中 $c[x]$ 保存 **序列 $a[]$ 的区间 $[x - \text{lowbit}(x) + 1, x]$ 中所有数的和**，即：

$$c[x] = \sum_{i=x-\text{lowbit}(x)+1}^x a[i]$$

事实上，数组 c 可以看作一个如下图所示的 **树形结构**，图的最下边一行是 n 个叶结点 ($n = 8$)，代表数值 $a[1 \sim n]$ 。(如果 n 不是 2 的整次幂，那么树状数组就是一个具有同样性质的 **森林结构**)



该结构满足以下性质：

1. 每个内部结点 $c[x]$ 保存以它为根的子树中所有叶结点的和，

$$c[x] = \sum_{i=x-\text{lowbit}(x)+1}^x a[i]。$$
2. 每个内部结点 $c[x]$ 的子结点个数等于 $\text{lowbit}(x)$ 的位数。
3. 除树根外，每个内部结点 $c[x]$ 的 **父结点** 是 $c[x + \text{lowbit}(x)]$ 。
4. 树的深度为 $O(\log N)$ 。

1. 求 $\text{lowbit}(n)$

lowbit 操作是树状数组的最基础的操作，也是为什么我们说树状数组中利用了二进制唯一分解的原因（实现上同时利用了负数的补码存储机制）。

$\text{lowbit}(n)$ 的含义是：取出非负整数 n 在二进制表示下最低位的 1 以及它后边的 0 构成的数值（如果记为第 k 位，那么就是求 2^k ），其代码实现如下：

```

int lowbit(int x)
{
    return x & (-x);
}
  
```

具体过程与原理分析如下：

设 $n > 0$ ， n 的第 k 位是 1，第 $0 \sim k - 1$ 位都是 0。

为了实现 `lowbit` 运算，先把 n 取反，此时第 k 位变为 0，第 $0 \sim k-1$ 位都是 1。再令 $n = n + 1$ ，此时因为进位，第 k 位变为 1，第 $0 \sim k-1$ 位都是 0。在上面的取反加 1 操作后， n 的第 $k+1$ 到最高位恰好与原来相反，所以 $n \& (\sim n + 1)$ 仅有第 k 位为 1，其余位都是 0。而在 **补码** 表示下， $-n = \sim n + 1$ ，因此 $\text{lowbit}(n) = n \& (\sim n + 1) = n \& (-n)$ 。

2. 对某个元素进行加法操作

树状数组支持单点增加，意思是给序列中的某个数 $a[x]$ 加上 y ，同时正确维护序列的前缀和。

根据上面给出的树形结构和它的性质，只有结点 $c[x]$ 及其 **所有祖先结点** 保存的“区间和”包含 $a[x]$ ，而任意一个结点的祖先至多只有 $\log n$ 个，我们逐一对它们的数组 c 值进行更新即可。

下面的代码在 $O(\log n)$ 时间内执行单点增加操作。

```
void add(int x, int y){
    while(x <= n) {
        c[x] += y;
        x += lowbit(x);
    }
}
```

特别说明： 对于 c 数组的初始化，为了简便，一般将 c 数组初始化为 0，并且在输入 $a[i]$ 的时候执行上面的 `add(i, a[i])` 操作，以此来实现 c 数组的初始化。

3. 查询前缀和

树状数组支持查询前缀和，即序列 a 第 $1 \sim x$ 个数的和。按照我们刚才提出的方法，应该对 x 进行二进制拆分，求出 x 的二进制表示中每个等于 1 的位，把 $[1, x]$ 分成 $O(\log n)$ 个小区间，而每个小区间的区间和都已经保存在数组 c 中。

下面的代码在 $O(\log n)$ 时间复杂度下查询前缀和：

```
int sum(int x) {
    int res = 0;
    while(x) {
        res += c[x];
        x -= lowbit(x);
    }
    return res;
}
```

4. 查询区间和

调用以上的查询前缀和操作，就可以在 $O(\log n)$ 时间复杂度下查询区间和：

```
// 求 a[x~y] 的和
int res = sum(y) - sum(x-1);
```

注意事项： 树状数组的下标不能从 0 开始使用，因为 $lowbit(0) = 0$ ，无法处理 0，会导致程序死循环。

5. 区间修改，单点查询

区间修改： 把 $x \sim y$ 区间内的所有值全部加上 k 或者减去 k 。

单点查询： 询问某个点的值。

这种时候该怎么做呢。如果用上面的树状数组，就必须把 $x \sim y$ 区间内每个值都更新，这样的时间复杂度肯定是不行的。所以这个时候，就不能再用数据的值建树了，这里我们引入差分，利用差分建树。**差分可以将区间修改转化为（两个）单点修改，将单点查询转化为查询前缀和。** 举例如下：

假设我们规定 $a[0] = 0$;

则有 $a[i] = \sum_{j=1}^i D[j]$, ($D[j] = a[j] - a[j-1]$) 即前面 i 项的差值和，这个有什么用呢？例如对于下面这个数组

- $a[] = 1\ 2\ 3\ 5\ 6\ 9$
- $D[] = 1\ 1\ 1\ 2\ 1\ 3$

如果我们把 $[2, 5]$ 区间内值加上 2，则变成了

- $a[] = 1\ 4\ 5\ 7\ 8\ 9$
- $D[] = 1\ 3\ 1\ 2\ 1\ 1$

发现了没有，当某个区间 $[x, y]$ 值改变了，区间内的差值是不变的，只有 $D[x]$ 和 $D[y + 1]$ 的值发生改变。

这样，原来要更新一个区间的值变成了只需要更新两个点，如此转化之后，就好处理了。

所以我们就可以利用这个性质对 $D[]$ 数组建立树状数组。

```
const int N = 1e6 + 5;
long long n, q, a[N], c[N];

long long lowbit(int x){
    return x&-x;
}

void add(int x, int y) {    // D[x] += y
    while(x<=n) {
        c[x] += y;
        x += lowbit(x);
    }
}

long long sum(int x) {    // 求D[1 ~ i]的和，即 a[i] 值
    long long res = 0;
    while(x) {
        res += c[x];
        x -= lowbit(x);
    }
    return res;
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> q;
    for(int i = 1; i <= n; i++) {
        cin >> a[i];
        add(i, a[i]-a[i-1]);    //输入初值的时候，也相当于更新了值，
        //同时构建的是差分数组上的树状数组
    }
}
```

```

while(q--){
    int op, x, y, z;
    cin >> op >> x;
    if(op == 1) {
        cin >> y >> z;
        add(x, z);          //a[x] - a[x-1] 增加 k
        add(y+1, -z);       //a[y+1] - a[y] 减少 k
    } else {
        cout << sum(x) << '\n';    // 差分数组上的前缀和就是原
        数组 a[x]
    }
}
}

```

6. 区间修改，区间查询

上面我们说的用差值建树状数组，得到的是某个点的值，那如果我们既要区间更新，又要区间查询怎么办。这里我们还是利用差分，由差分数组的定义可知：

$$\begin{aligned}
 sum_{i=1}^n a[i] &= \sum_{i=1}^n \sum_{j=1}^i D[j] \\
 &= D[1] + (D[1] + D[2]) + \dots + (D[1] + D[2] + \dots + D[n]) \\
 &= n * D[1] + (n-1) * D[2] + \dots + D[n] \\
 &= n * (D[1] + D[2] + \dots + D[n]) - (0 * D[1] + 1 * D[2] + \dots + (n-1) * D[n]) \\
 &= n * \sum_{i=1}^n D[i] - \sum_{i=1}^n (i-1) * D[i]
 \end{aligned}$$

同单点修改，我们只需要开两个树状数组，一个维护 $D[i]$ ，一个维护 $(i-1) * D[i]$ 就行了。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1e6+5;
int n, q, a[N];
long long c1[N];    // 维护 (D[1] + D[2] + ... + D[n])
long long c2[N];    // 维护 (0*D[1] + 1*D[2] + ... + (n-1)*D[n])

```

```

// 获取 x 的最低位 1 的位置
int lowbit(int x) {
    return x & -x;
}

// 在树状数组中进行单点更新
void add(int x, int y) {
    long long p = x; // 因为 x 不变，所以要先保存
    while(x <= n) {
        c1[x] += y;
        c2[x] += (p - 1) * y;
        x += lowbit(x);
    }
}

// 计算前缀和
long long sum(int x) {
    long long res = 0, p = x;
    while(x) {
        res += p * c1[x] - c2[x];
        x -= lowbit(x);
    }
    return res;
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    // 输入数组长度和查询次数
    cin >> n >> q;

    // 输入数组元素
    for(int i = 1; i <= n; i++) {
        cin >> a[i];
        add(i, a[i] - a[i - 1]);
    }

    while(q--) {

```



```

int op, l, r, x;
cin >> op >> l >> r;
if(op == 1) {
    cin >> x;
    add(l, x);
    add(r + 1, -x);
} else {
    cout << sum(r) - sum(l - 1) << '\n';
}

return 0;
}

```

ST表

专门解决多次询问一些区间的最值的问题的算法。

简介

max = 14							
max = 14							
max = 13							
max = 0							
0	13	14	4	13	1	5	7

ST 表是用于解决 **可重复贡献问题** 的数据结构。

什么是可重复贡献问题？

可重复贡献问题 是指对于运算 opt ，满足 $x \text{ opt } x = x$ ，则对应的区间询问就是一个可重复贡献问题。例如，最大值有 $\max(x, x) = x$ ，gcd 有 $\gcd(x, x) = x$ ，所以 RMQ 和区间 GCD 就是一个可重复贡献问题。像区间和就不具有这个性质，如果求区间和的时候采用的预处理区间重叠了，则会导致重叠部分被计算两次，这是我们所不愿意看到的。另外， opt 还必须满足结合律才能使用 ST 表求解。

常见的可重复贡献问题有：最大值、最小值、最大公因数、最小公倍数、按位或、按位与都符合这个条件。

可重复贡献的意义在于，可以对两个交集不为空的区间进行信息合并。

RMQ 问题

以求区间最大值为例，给定 n 个数，有 m 个询问，对于每个询问，你需要回答区间 $[l, r]$ 中的最大值。

考虑朴素的暴力做法。每次都对区间 $[l, r]$ 扫描一遍，求出最大值。

显然，这个算法时间复杂度是 $O(nm)$ 会超时。

ST 表

ST 表是基于 **倍增** 思想，可以做到 $O(n \log n)$ 预处理， $O(1)$ 回答每个询问。但是不支持修改操作。

基于倍增思想，我们考虑如何求出区间最大值。可以发现，如果按照一般的倍增流程，每次跳 2^i 步的话，询问时的复杂度仍旧是 $O(\log n)$ ，并没有比线段树更优，反而预处理一步还比线段树慢。

再深入思考，我们发现 $\max(x, x) = x$ ，也就是说，区间最大值是一个具有**可重复贡献**性质的问题。即使用来求解的预处理区间有重叠部分，只要这些区间的并是所求的区间，最终计算出的答案就是正确的。

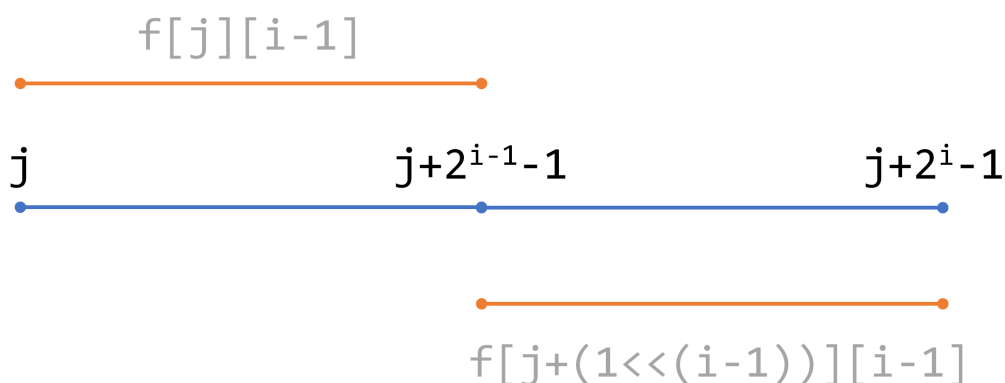
具体实现如下：

1. 预处理

令 $f[i][j]$ 表示区间 $[i, i + 2^j - 1]$ 的最大值。

显然 $f[i][0] = a_i$ 。

根据定义式，第二维就相当于倍增的时候“跳了 $2^j - 1$ 步”，依据倍增的思路，写出状态转移方程： $f(i, j) = \max(f(i, j - 1), f(i + 2^{j-1}, j - 1))$ 。

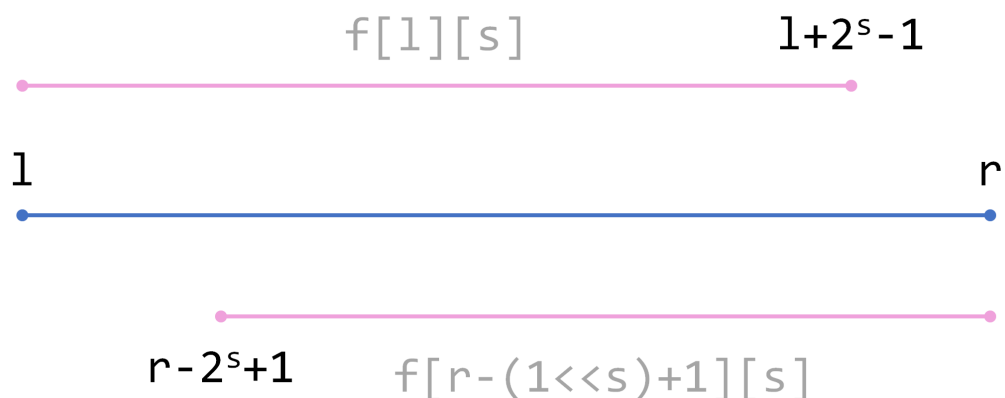


为了减少时间复杂度，可以用 **动态规划** 的方法进行 **预处理**：

```
int f[N][21]; // 第二维的大小根据数据范围决定，不小于log(N)
for (int i = 1; i <= n; ++i)
    cin >> f[i][0]; // 读入数据 f[i][0] = a[i];
for (int i = 1; i <= 20; ++i)
    for (int j = 1; j + (1 << i) - 1 <= n; ++j)
        f[j][i] = max(f[j][i - 1], f[j + (1 << (i - 1))][i - 1]);
```

手动模拟一下，可以发现我们能使用至多两个预处理过的区间来覆盖询问区间，也就是说询问时的时间复杂度可以被降至 $O(1)$ ，在处理有大量询问的题目时十分有效。

查询时，我们需要找到两个 $[l, r]$ 的子区间，它们的并集恰是 $[l, r]$ （不必不相交）。具体地说，我们要找的是一个整数 s ，两个子区间分别为 $[l, l + 2^s - 1]$ 和 $[r - 2^s + 1, r]$ 。（如下图）



我们希望前一个子区间的右端点尽可能接近 r 。当 $l + 2^s - 1 = r$ 时，有 $s = \log_2(r - l + 1)$ （这时 $r - 2^s + 1 = l$ 也成立）。

但因为 s 是整数，所以我们取 $s = \lfloor \log_2(r - l + 1) \rfloor$ 。可以证明，**这时两个子区间确实可以覆盖 $[l, r]$** 。

每次计算 $\log_2 i$ 太花时间了，我们可以对 $\log_2 i$ 也进行一次递推的预处理：

```
for (int i = 2; i <= n; ++i)
    logn[i] = logn[i / 2] + 1;
```

在线查询的代码如下：

```
for (int i = 1; i <= m; ++i)
{
    cin >> l >> r;
    int s = logn[r - l + 1];
    cout << max(f[l][s], f[r - (1 << s) + 1][s]) << '\n';
}
```

根据上面对于“可重复贡献问题”的论证，由于最大值是“可重复贡献问题”，重叠并不会对区间最大值产生影响。又因为这两个区间完全覆盖了 $[l, r]$ ，可以保证答案的正确性。

单次查询的时间复杂度很明显是 $O(1)$ 的，效率很高。

模板代码

带快读的ST表

```
#include <iostream>
#include <cstdio>
using namespace std;

const int N = 1e5+5, logN = 21;    // 2^21 = 2097152
int n, m, s, l, r;
int f[N][logN], logn[N];

// f[i][j]: 从 i 开始，长度是 2^j 的区间[i, i+2^j-1]的最大值
// f[i][j] = max(f[i][j-1], f[i+1<<(j-1)][j-1])
// l, r:    s = logn(r-l+1)
// f[l][r] = max(f[l][s], f[r-(1<<s)+1][s])

inline int read()
```

```

{
    int x = 0, f = 1; char ch = getchar();
    while(!isdigit(ch)){
        if(ch == '-') f = -1;
        ch = getchar();
    }
    while(isdigit(ch)){
        x = x*10 + ch - 48;
        ch = getchar();
    }
    return x*f;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    // cin >> n >> m;
    n = read(); m = read();
    for(int i = 1; i <= n; i++) f[i][0] = read();

    // 打表:  $O(n \log n)$ 
    logn[2] = 1;
    for(int i = 3; i <= n; i++)
        logn[i] = logn[i/2] + 1;
    for(int j = 1; j <= logN; j++)
        for(int i = 1; i+(1<<(j-1)) <= n; i++)
            f[i][j] = max(f[i][j-1], f[i+(1<<(j-1))][j-1]);

    // 查询:  $m \cdot O(1)$ 
    for(int i = 1; i <= m; i++)
    {
        l = read(); r = read();
        s = logn[r-l+1];
        cout << max(f[l][s], f[r-(1<<s)+1][s]) << '\n';
    }
    return 0;
}

```

蓝桥杯真题

① Note

因为树状数组和线段树基本使用场景一致，前面也已经出了线段树的国赛真题，于是下面又找了两个适合练习的省赛真题。前面的线段树模块也可以用树状数组去写。

小朋友排队（蓝桥杯C/C++2014B组省赛）

对于一个数字，他至少要跟前面比他小的交换，也至少要跟后面比他大的交换。这是它最小的交换次数。

那么我们怎么快速求出前面比他小的数和后面比他大的数呢？

直接用树状数组跑两遍即可，类似逆序对。

时间复杂度为 $O(n\log n)$

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int N=1e6+114;

int a[N], s[N];
int sum[N];
int n;

int lbit (int x) {
    return x & -x;
}

int query (int x) {
    int res=0;
    for(int i=x;i;i-=lbit (i)) res+=s[i];
    return res;
}

void auid (int x,int v) {
    for(int i=x;i<N;i+=lbit (i)) s[i]+=v;
```

```

}
signed main()
{
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>a[i];
        a[i]++;
    }

    for(int i=1;i<=n;i++) {
        sum[i]=query(N-1)-query(a[i]);
        auid (a[i],1);
    }

    memset (s,0,sizeof (s));
    for(int i=n;i>0;i--){
        sum[i]+=query (a[i]-1);
        auid (a[i],1);
    }

    ll as=0;
    for(int i=1;i<=n;i++) as+=(ll)sum[i]*(sum[i]+1)/2;
    cout<<as;
    return 0;
}

```

压缩变换（蓝桥杯Java2016B组省赛）

可以发现我们求的是不同数的个数，那么只需要再相应的下标上面做数量的修改即可。

也就是说如果这个数没有出现过，那么当前下标加1，否则先计算一下之前的下标之后和当前位置之前有几个1，然后再把之前下标位置设置为0，就是之前的位置加-1，然后把现在位置又加1。

再更新一下当前数的位置即可。

所用的是动态的单点修改，区间查询，线段树树状数组都可以，但是树状数组代码短好写运行还快，所以示例代码用的树状数组。

```

#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 10;
unordered_map<int,int> bk;
int t[N];
void add(int x,int y)
{
    for(int i=x;i<N;i+=i&(-i)) t[i]+=y;
}
int sum(int x)
{
    int s=0;
    for(int i=x;i>=1;i-=i&(-i)) s+=t[i];
    return s;
}
int main()
{
    int n,x;
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>x;
        if(!bk[x])
        {
            cout<<-x<<" ";
            bk[x]=i;
            add(i,1);
        }
        else
        {
            cout<<sum(i)-sum(bk[x])<<" ";
            add(bk[x],-1);
            bk[x]=i;
            add(i,1);
        }
    }
}

```


习题

求逆序对

树状数组求逆序对

任意给定一个集合 a ，如果用 $t[val]$ 保存数值 val 在集合 a 中出现的次数，那么数组 t 在 $[l, r]$ 上的区间和（即 $\sum_{i=l}^r t[i]$ ）就表示集合 a 中范围在 $[l, r]$ 内的数有多少个。

我们可以在集合 a 的数值范围上建立一个树状数组，来维护 t 的前缀和。这样即使在集合 a 中插入或删除一个数，也可以高效地进行统计。

我们前面已经学习过了逆序对问题以及使用归并排序的解法。对于一个序列 a ，若 $i < j$ 且 $a[i] > a[j]$ ，则称 $a[i]$ 与 $a[j]$ 构成逆序对。按照上述思路，利用树状数组也可以求出一个序列的逆序对个数：

1. 在序列 a 的数值范围上建立树状数组，初始化为全零。
2. 倒序扫描给定的序列 a ，对于每个数 $a[i]$ ：
 - (1) 在树状数组中查询前缀和 $[1, a[i] - 1]$ ，累加到答案 ans 中。
 - (2) 执行“单点增加”操作，即把位置 $a[i]$ 上的数加 1（相当于 $t[a[i]]++$ ），同时正确维护 t 的前缀和。这表示数值 $a[i]$ 又出现了 1 次。
3. ans 即为所求。

```
for(int i = n; i; i--) {  
    ans += sum(a[i]-1);  
    add(a[i], 1);  
}
```

在这个算法中，因为倒序扫描，“已经出现过的数”就是在 $a[i]$ 后边的数，所以我们通过树状数组查询的内容就是“每个 $a[i]$ 后边有多少个比它小”。每次查询的结果之和当然就是逆序对个数。时间复杂度为 $O((N + M) \log M)$ ， M 为数值范围的大小。

当数值范围较大时，当然可以先进行离散化，再用树状数组进行计算。不过因为离散化本身就要通过排序来实现，所以在这种情况下就不如直接用归并排序来计算逆序对数了。

```
#include <bits/stdc++.h>  
using namespace std;
```

```

const int N = 1e6 + 5;
int n, t, b[N], c[N], cnt;
long long ans;
pair<int, int> a[N];
int lowbit(int x){
    return x&-x;
}
void add(int x, int y) {
    while(x<=n) {
        c[x] += y;
        x += lowbit(x);
    }
}
int sum(int x) {
    int res = 0;
    while(x) {
        res += c[x];
        x -= lowbit(x);
    }
    return res;
}
int main(){
    cin >> n;
    for(int i = 1; i <= n; i++) {
        cin >> a[i].first;
        a[i].second = i;
    }
    sort(a+1, a+n+1);
    // b[i]: 原数组元素 a[i] 是原数组去重离散化后第 b[i] 大的数字
    cnt = 1; b[a[1].second] = cnt;
    for(int i = 2; i <= n; i++) {
        if(a[i].first != a[i-1].first)
            cnt++;
        b[a[i].second] = cnt;
    }
    for(int i = n; i; i--) { // 倒序扫描
        ans += sum(b[i]-1); // 比 a[i] 小的数字的数量
        add(b[i], 1);
    }
}

```

```

    }
    cout << ans;
}

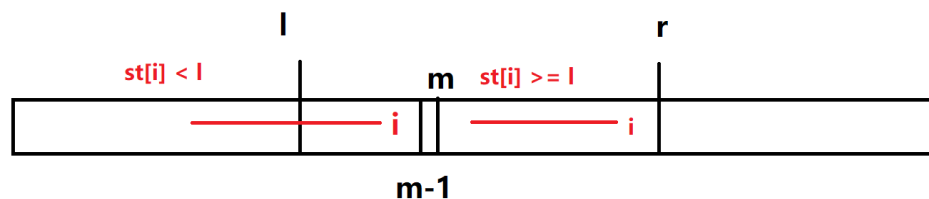
```

与众不同

本题是 RMQ 较复杂的应用。用 $last[value]$ 表示盈利值 $value$ 最近出现的位置，一开始全部赋为 0。

用 $st[i]$ 表示以第 i 个数为结尾的最长完美序列的起始位置， st 的计算可以用以下递推式得到： $st[i] = \max(st[i-1], last[a[i]] + 1)$ ($a[i]$ 表示第 i 个月的盈利值)。表示以第 i 个数为结尾的完美序列的起始位置必须在 $last[a[i]]$ 之后。用 $f[i]$ 表示第 i 个数为结尾的最长完美序列的长度，很显然， $f[i] = i - st[i] + 1$ ， $st[i]$ 和 $f[i]$ 都可以在 $O(n)$ 内计算出来。

由 st 的递推式可知， st 的值是一个非递减的序列，那么对于一个询问区间 $[l, r]$ ，该区间内的 st 值可能会出现：左边一部分的 st 值不在区间内，即小于 l ，而右边一部分的 st 值大于等于 l ，由于 st 值具有单调性，所以这个分界点可以通过二分得到，假设该位置求出来为 m ，即 $st[l \sim m-1] < l$ ，而 $st[m \sim r] \geq l$ ，如下图所示。



那么整个区间 $[l, r]$ 的最长完美序列的长度，分两部分来求，其中左边部分的最大值根据 st 的单调性，很显然为 $m - l$ ，而右边一部分由于 st 值大于等于 l ，所以就是求 $[m, r]$ 内的最大值，可以用 ST 算法来求，所以整个问题的时间复杂度为 $O((M + N) \log N)$ ，分别是 ST 算法预处理的时间和二分询问的时间。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 2e5+5, M = 1e6+5;
int n, l, r, m, x;
int st[N][20], logn[N];
int last[M<<1]; // last[i]: 数字 i 最近出现的位置, 初始都是 0
int s[N]; // 以第 i 个数字为结尾的最长完美序列的起始位置

```

```

int f[N]; // 以第 i 个数字为结尾的最长完美序列的长度

int find(int li, int ri) {
    if(s[li] == li) return li;
    if(s[ri] < li) return ri + 1;
    int fl = li, fr = ri, mid, ans;
    while(fl <= fr) {
        mid = (fl+fr)/2;
        if(s[mid] < li) {
            fl = mid+1;
        }
        else fr = mid-1;
    }
    return fl;
}

int query(int li, int ri) {
    int s = logn[ri - li + 1];
    int x = ri - (1<<s) + 1;
    return max(st[li][s], st[x][s]);
}

int main(){
    cin >> n >> m;
    logn[0] = -1;
    for(int i = 1; i <= n; i++) {
        cin >> x; // a[i];
        s[i] = max(s[i-1], last[x+M]+1);
        f[i] = i - s[i] + 1;
        last[x+M] = i;
        logn[i] = logn[i/2] + 1;
        st[i][0] = f[i]; // st[i][j]: i~(i+2^j-1) 最长完美序列长度
    }
    for(int i = 1; i <= 20; i++) {
        for(int j = 1; j + (1<<i)-1 <= n; j++) {
            st[j][i] = max(st[j][i-1], st[j+(1<<(i-1))][i-1]);
        }
    }
}

```

```

while(m--) {
    cin >> l >> r;
    l++; r++;
    int mi = find(l, r);
    int ans = 0, t;
    if(mi > l) ans = mi - l;
    if(mi <= r) {
        t = query(mi, r); // 查询 mi~r 中 f 最大值
        ans = max(ans, t);
    }
    cout << ans << '\n';
}
return 0;
}

```